



FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

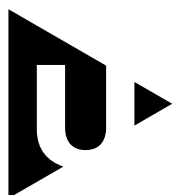
KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY



VR aplikace pro prezentaci výsledků výzkumu editace sekvencí polygonálních sítí

David Konečný





FAKULTA APLIKOVANÝCH VĚD
ZÁPADOČESKÉ UNIVERZITY
V PLZNI

KATEDRA INFORMATIKY
A VÝPOČETNÍ TECHNIKY

VR aplikace pro prezentaci výsledků výzkumu editace sekvencí polygonálních sítí

David Konečný

© David Konečný, 2026

Všechna práva vyhrazena. Žádná část tohoto dokumentu nesmí být reprodukována ani rozšiřována jakoukoli formou, elektronicky či mechanicky, fotokopírováním, nahráváním nebo jiným způsobem, nebo uložena v systému pro ukládání a vyhledávání informací bez písemného souhlasu držitelů autorských práv.

Citace v seznamu literatury:

KONEČNÝ, David. *VR aplikace pro prezentaci výsledků výzkumu editace sekvencí polygonálních sítí*. Plzeň, 2026. Dokumentace k bakalářskému projektu. Západočeská univerzita v Plzni, Fakulta aplikovaných věd, Katedra informatiky a výpočetní techniky.

Obsah

1	Úvod	1
2	Výchozí stav a cíle práce	2
2.1	Zadání práce	2
2.2	Kontext existujících projektů	2
2.3	Hlavní cíl práce	2
2.4	Dílčí cíle	3
2.5	Shrnutí výchozího stavu	3
3	Návrh řešení	4
3.1	Základní princip řešení	4
3.2	Rozdělení projektu do vrstev	5
3.3	Request a runtime kontext	6
3.4	Společné výpočetní jádro a napojení metod	6
3.5	Návrh vstupu pro Inflate/Deflate	7
3.6	Shrnutí návrhu	8
4	Implementace	9
4.1	Migrace projektu a aktualizace prostředí	9
4.2	Klientská VR vrstva	10
4.2.1	Načítání sekvence a runtime stav	10
4.2.2	Interakce s centry	10
4.2.3	Ovládací panely a uživatelský workflow	11
4.3	Datové rozhraní mezi klientem a výpočtem	18
4.3.1	Request a výsledek operace	18
4.3.2	Runtime kontext sekvence	19
4.4	Core vrstva a editační metody	19
4.4.1	Basic Translate	20
4.4.2	Inflate/Deflate	20
4.5	Sběr efektorů pro Inflate/Deflate	21
4.6	Cache	22

5	Výsledky a omezení	24
5.1	Výsledky implementace	24
5.2	Ověření chování aplikace a uživatelského workflow	24
5.3	Omezení aktuálního řešení	25
5.4	Použití AI při práci	25
6	Závěr	26
	Seznam použité literatury	27

V čase proměnlivé sekvence polygonálních sítí představují specifický typ 3D dat, u kterého je nutné pracovat nejen s podobou jednotlivého objektu, ale i s jeho vývojem v čase. Na rozdíl od běžných animovaných modelů nemají tyto sekvence konzistentní konektivitu ani konzistentní počet vrcholů. To výrazně omezuje použití běžných editačních metod a komplikuje i návrh uživatelského ovládání. Virtuální realita je pro tento typ úloh vhodným prostředím, protože umožňuje prostorovou interakci přímo nad zobrazovaným objektem.

Tato práce navazuje na dva existující projekty. Prvním z nich je původní VR aplikace „tvm-VR“ [1], která poskytovala základní prostředí pro načítání, zobrazení a editaci těchto sekvencí. Druhým je výpočetně orientovaný projekt „tvm-editing-master“ [2], jenž obsahuje pokročilejší metody deformace. Ukázalo se, že pro další rozvoj aplikace nestačí pouze rozšiřovat původní klientské řešení, ale je nutné navrhnout novou architekturu, která umožní bezpečně propojit VR prostředí s více výpočetními metodami.

Práce je rozdělena do částí věnovaných výchozímu stavu, návrhu řešení, implementaci a zhodnocení dosažených výsledků. Hlavně bude popsána architektura projektu „tvm-VR-2“, implementace jeho hlavních částí a integrace metody Inflate/Deflate do VR prostředí.

Výchozí stav a cíle práce

2

2.1 Zadání práce

Téma práce se zabývá rozšířením existující VR aplikace pro editaci sekvencí časově proměnných trojúhelníkových sítí. Cílem je doplnit do VR editoru další editační metody dostupné v desktopové aplikaci, vytvořit k nim odpovídající ovládací prvky a upravit workflow tak, aby aplikace mohla být bez provozních překážek využita i při živých ukázkách, například na dni otevřených dveří.

2.2 Kontext existujících projektů

Práce vychází ze dvou navazujících projektů. Původní projekt „tvm-VR“ [1] představuje VR editor, který zajišťuje klientské prostředí, načítání sekvencí a základní interakci s centry. Projekt „tvm-editing-master“ [2] obsahuje pokročilejší výpočetní metody, zejména deformaci založenou na Inflate/Deflate přístupu.

Ukázalo se, že pro další rozvoj aplikace nestačí pouze rozšiřovat původní klientské řešení. Je potřeba nová architektura, která dovolí bezpečně propojit VR prostředí s více výpočetními metodami a současně zachovat přehledné uživatelské ovládání.

2.3 Hlavní cíl práce

Cílem práce je navrhnout a implementovat projekt „tvm-VR-2“ jako VR aplikaci pro prezentaci výsledků výzkumu editace sekvencí polygonálních sítí. Prakticky to znamená zpřístupnit vybrané metody úprav uživateli ve VR prostředí, navrhnout přehlednější architekturu aplikace, oddělit klientskou vrstvu od výpočetního jádra a vytvořit takový základ, do kterého lze vhodně napojit další editační metody.

2.4 Dílčí cíle

Mezi dílčí cíle patří převod základní metody z původního editoru do nové architektury, integrace metody Inflate/Deflate z externího projektu, návrh sběru vstupu a tvorby requestu pro obě metody, úprava klientské VR vrstvy a ověření, že finální návrh zůstává ve VR prostředí prakticky použitelný.

2.5 Shrnutí výchozího stavu

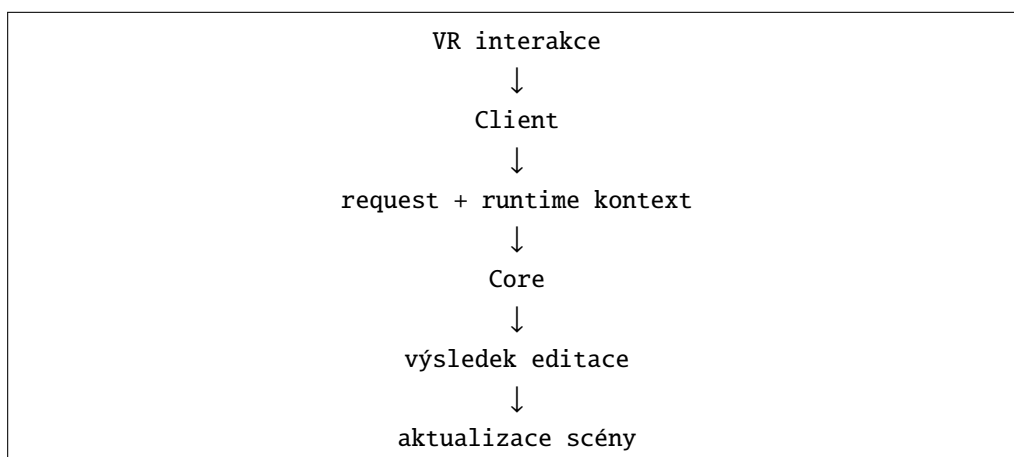
Výchozí stav je použitelný jako zdroj existujících částí, ale ne jako finální základ pro další rozvoj. Původní editor poskytoval funkční VR prostředí, zatímco výpočetní projekt poskytoval vhodný algoritmický základ pro pokročilejší deformaci. Chyběla však architektura, která by klientské a výpočetní odpovědnosti oddělila a současně dovolila jejich praktické propojení. Z tohoto důvodu vznikl projekt „tvm-VR-2“ jako nová integrační větev.

3.1 Základní princip řešení

Navržené řešení vychází z jednoduchého principu: VR aplikace sbírá uživatelský vstup, převádí jej na datový požadavek, dále označovaný jako request, předává jej výpočetnímu jádru a po dokončení výpočtu promítá výsledek zpět do scény. Tím se odděluje interakce s uživatelem od samotné deformace objektu a současně se vytváří společný způsob, jak do aplikace připojovat více metod.

Z pohledu průchodu jedné editace návrh počítá s těmito kroky:

1. uživatel provede akci ve VR prostředí,
2. klientská vrstva vytvoří request odpovídající zvolené metodě,
3. request je spolu s runtime kontextem předán do výpočetního jádra,
4. Core vybere odpovídající metodu a provede výpočet,
5. výsledek je vrácen zpět klientské vrstvě,
6. klientská vrstva aktualizuje centra, mesh a stav aplikace.



Obrázek 3.1 základní tok jedné editační operace v navrženém řešení.

Zdroj: vlastní zpracování.

3.2 Rozdělení projektu do vrstev

Aktivní část projektu byla rozdělena do tří vrstev: *Client*, *Api* a *Core*. Každá z nich nese jiný typ odpovědnosti. Smyslem tohoto rozdělení není vytvářet složitou architekturu, ale prakticky oddělit scénu a uživatelské ovládání od výpočtu.

vrstva	role
vrstva <i>Client</i>	sběr VR vstupu, práce se scénou, zobrazení center a meshe, klientské workflow
vrstva <i>Api</i>	requesty, výsledky operací, výčtové typy metod a runtime kontext
vrstva <i>Core</i>	výběr metody, převod vstupů a vlastní výpočet deformace

Tabulka 3.1 základní role tří hlavních vrstev.

Rozdělení do vrstev bylo současně promítnuto i do fyzické struktury zdrojových souborů. U každé vrstvy je tak z projektu patrné, které části zajišťují klientské VR chování, které tvoří datovou hranici a které nesou vlastní výpočetní logiku.

Fyzická struktura hlavních částí projektu je soustředěna v adresáři se zdrojovými skripty. Zjednodušený přehled nejdůležitějších částí je uveden níže: Přehled hlavních částí je shrnut na obrázku 3.2.

Client/

- *Sequence*/ – načítání sekvence a řízení workflow
- *Centers*/ – správa a interakce center
- *UI*/ – ovládací prvky editačních metod
- *Sequence.cs*, *CenterPool.cs*, *InflateDeflateUI.cs*

Api/

- *Requests*/ – vstupní datové požadavky
- *Responses*/ – výsledky operací
- *Sequence*/ – runtime kontext sekvence
- *InflateDeflateRequest.cs*, *EditOperationResult.cs*, *SequenceRuntimeContext.cs*

Core/

- *EditingCore.cs* – společný vstup do výpočtu
- *Methods/BasicTranslate*/ – metoda převzatá z původního editoru
- *Methods/InflateDeflate*/ – napojení metody z externího projektu
- *BasicTranslatePipeline.cs*, *InflateDeflateMethod.cs*, *TvmEditingMasterInflateDeflateAdapter.cs*

Obrázek 3.2 zjednodušený přehled hlavních částí ve složce Assets/Scripts/.

Zdroj: vlastní zpracování.

Přehled neuvádí všechny podsložky ani všechny soubory projektu. Slouží pouze k zachycení hlavních částí, které jsou důležité pro návrh nové architektury a integraci editačních metod.

3.3 Request a runtime context

Mezi klientem a výpočtem byla zavedena explicitní datová hranice. Klientská vrstva nepředává výpočtu přímo objekty Unity scény, ale pouze data popisující zamýšlenou operaci. Tato data jsou rozdělena na dvě části:

- **request**, který nese uživatelský záměr,
- **runtime context**, který nese technická data potřebná pro výpočet.

část	obsah
request	volba metody, vybrané centrum nebo oblast, cílová pozice nebo parametry deformace
runtime kontext	identifikace sekvence, aktuální snímek, sekvenční data a běhová konfigurace metody

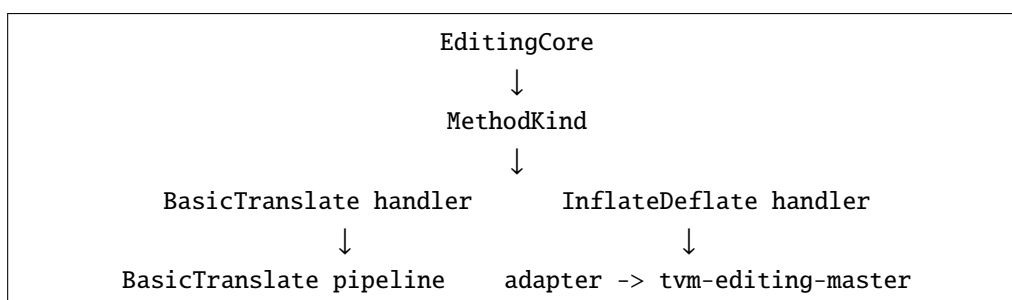
Tabulka 3.2 rozdělení vstupních dat mezi request a runtime kontext.

Tento návrh zajišťuje, že výpočetní vrstva není závislá na konkrétní scéně ani na konkrétních komponentách VR rozhraní. Současně je zřejmé, která data pocházejí přímo od uživatele a která slouží jen jako provozní kontext aplikace.

3.4 Společné výpočetní jádro a napojení metod

Všechny editační metody jsou připojeny přes společný vstupní bod `EditingCore`. Klientská vrstva tak nevolá jednotlivé algoritmy přímo, ale vždy komunikuje se stejným jádrem. To nejprve zkontroluje request, vybere odpovídající handler a předá data konkrétní metodě.

Základní vztah mezi jádrem a metodami zachycuje schéma 3.3.



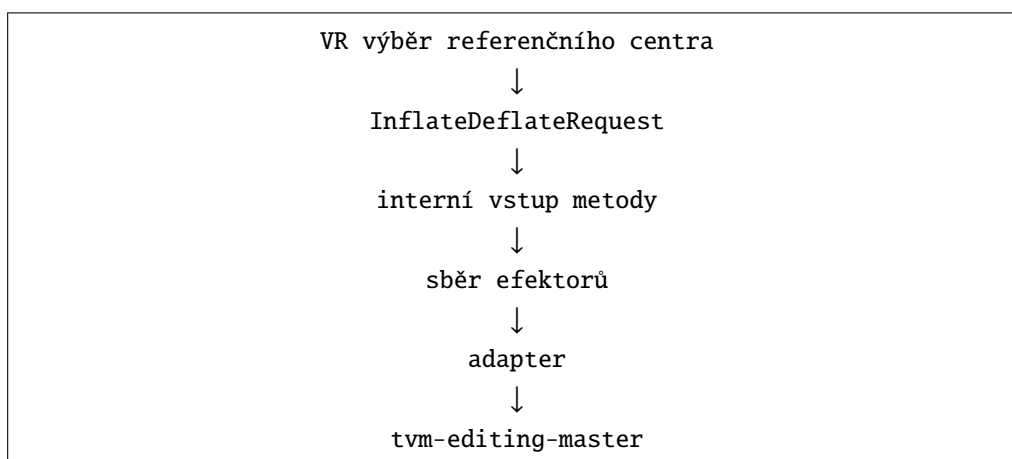
Obrázek 3.3 napojení jednotlivých metod přes společné výpočetní jádro.

Zdroj: vlastní zpracování.

U metody Basic Translate je návrh přímočařejší, protože jde o převod původní metody do nové architektury. U metody Inflate/Deflate bylo navíc nutné navrhnout adapter mezi interním vstupem aplikace a výpočetní reprezentací používanou projektem „tvm-editing-master“. Tento adapter zajišťuje, že metoda může být volána přes stejný princip jako ostatní, i když její výpočetní základ vznikl mimo původní VR editor.

3.5 Návrh vstupu pro Inflate/Deflate

Metoda Inflate/Deflate vyžaduje jiný typ vstupu než základní posun centra. Uživatel ve VR prostředí neurčuje cílovou pozici jednoho centra, ale vybírá referenční centrum, režim deformace a sílu zásahu. Návrh proto počítá s převodem tohoto vstupu do samostatného requestu a následně do interní reprezentace vhodné pro výpočetní metodu.



Obrázek 3.4 návrh převodu VR vstupu pro metodu Inflate/Deflate.

Zdroj: vlastní zpracování.

Důležitým návrhovým bodem je sběr efektorů. Veřejný request popisuje uživatelský záměr, tedy referenční centrum, režim operace a hodnotu *strength*. Výpočetní metoda však pracuje s interní sadou vstupů, které musí být z tohoto záměru odvozeny. Tato mezivrstva umožňuje, aby VR ovládání zůstalo jednoduché a aby zároveň bylo možné připravit data ve tvaru očekávaném externím výpočetním projektem.

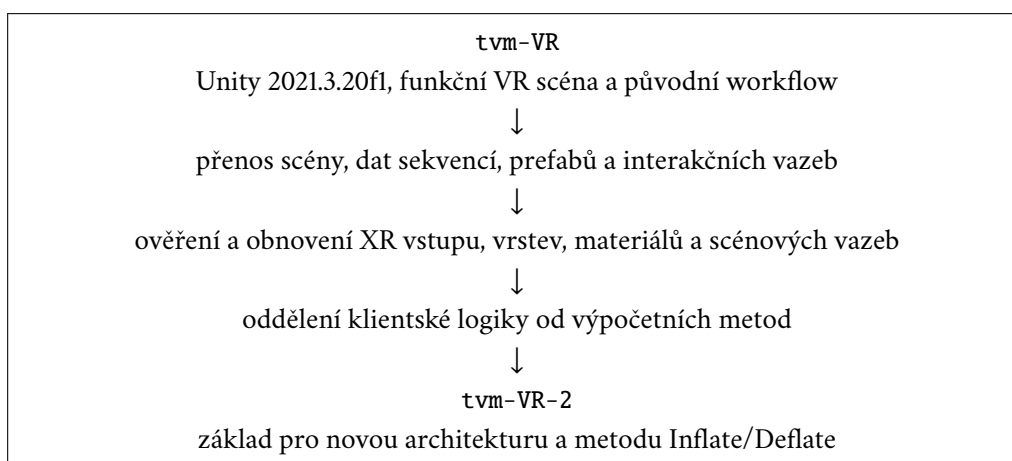
3.6 Shrnutí návrhu

Navržené řešení odděluje VR klienta od výpočtu, zavádí explicitní request model a připojuje jednotlivé metody přes společné výpočetní jádro. Tím vzniká architektura, která řeší hlavní problém původního stavu: umožňuje zachovat funkční VR prostředí a současně bezpečně napojit i metodu převzatou z externího projektu. V další kapitole bude ukázáno, jak byl tento návrh převeden do konkrétní implementace projektu „tvm-VR-2“.

4.1 Migrace projektu a aktualizace prostředí

Implementace projektu „tvm-VR-2“ začala převodem původní aplikace z Unity **2021.3.20f1** do Unity **6000.3.5f1**. Migrace nebyla samostatným cílem práce, ale nutným krokem pro další vývoj VR klienta a napojení nových editačních metod. Při převodu bylo potřeba zachovat funkční scénu, načítání sekvencí polygonálních sítí a základní VR interakci.

Převod byl proveden selektivně. Některé části byly pouze přeneseny a ověřeny v novější verzi Unity, například vrstvy scény, kolizní matice, část XRI infrastruktury a základní materiály. Skutečné implementační zásahy se týkaly hlavně obnovení scénových vazeb na vstupní akce, doplnění nových VR panelů a oddělení klientské logiky od výpočetní části.



Obrázek 4.1 hlavní tok migrace původního projektu do „tvm-VR-2“.

Zdroj: vlastní zpracování.

kategorie	příklad
přeneseno	scéna, data sekvencí, vrstvy, kolizní matice, část XRI infrastruktury a základní materiály
obnoveno	vazby levé a pravé ruky na input actions, runtime vazby ve scéně, reference UI panelů
doplněno	panel výběru metody, Inflate Deflate Panel, ovládání Apply, Cancel, Mode a Strength
nahrazeno	přímé napojení výpočtu na klientskou scénu, část odpovědností původní třídy Sequence

Tabulka 4.1 přehled hlavních rozhodnutí při migraci původního projektu.

Výsledkem migrace je funkční základ projektu v Unity **6000.3.5f1**, který zachovává původní VR scénu a práci se sekvencí, ale zároveň připravuje prostor pro nové klientské workflow a napojení metody Inflate/Deflate.

4.2 Klientská VR vrstva

Klientská vrstva je část aplikace, se kterou uživatel přímo pracuje ve VR. Zajišťuje načtení sekvence, zobrazení meshe a center, zpracování interakce rukou, zobrazení ovládacích panelů a předání uživatelského záměru do výpočetního jádra.

4.2.1 Načítání sekvence a runtime stav

Třída Sequence zůstává hlavním klientským koordinátorem běžící scény. Po refaktoringu však již neobsahuje vlastní výpočet deformace. Její odpovědností je načtení sekvence, držení aktuálního runtime stavu, sestavení requestu podle zvolené metody a aplikace výsledku zpět do scény.

Runtime stav tvoří hranici mezi daty sekvence a Unity scénou. Výpočetní jádro nepracuje přímo se scénovými objekty, ale s daty předanými přes request a runtime kontext.

4.2.2 Interakce s centry

Původní projekt používal centra jako interaktivní objekty pro hover a grab. V projektu „tvm-VR-2“ byla tato část zachována, ale rozšířena o vazbu na aktuálně zvolenou metodu. Třídy CenterPool a CenterUI proto již neřeší pouze vytvoření center a jejich výběr, ale také to, zda má být centrum uchopitelné, pouze vybrané jako referenční bod, nebo použité pro preview oblasti zásahu.

Interakce s centrem se spouští u obou metod stejným způsobem. Rozdíl je v tom, jak se vybraný objekt použije. U Basic Translate je povolen pohyb centra a jeho nová poloha slouží pro výpočet translace. U Inflate/Deflate se centrum neposouvá,

ale pouze označuje jako referenční bod deformace. Trvalé vizuální označení referenčního centra zajišťuje `SetPersistentSelected()`, náhled ovlivněné oblasti podle síly metody vytváří `PreviewInflateDeflate()` a zapínání nebo vypínání možnosti uchopení center řeší `SetInteractionEnabled()`.

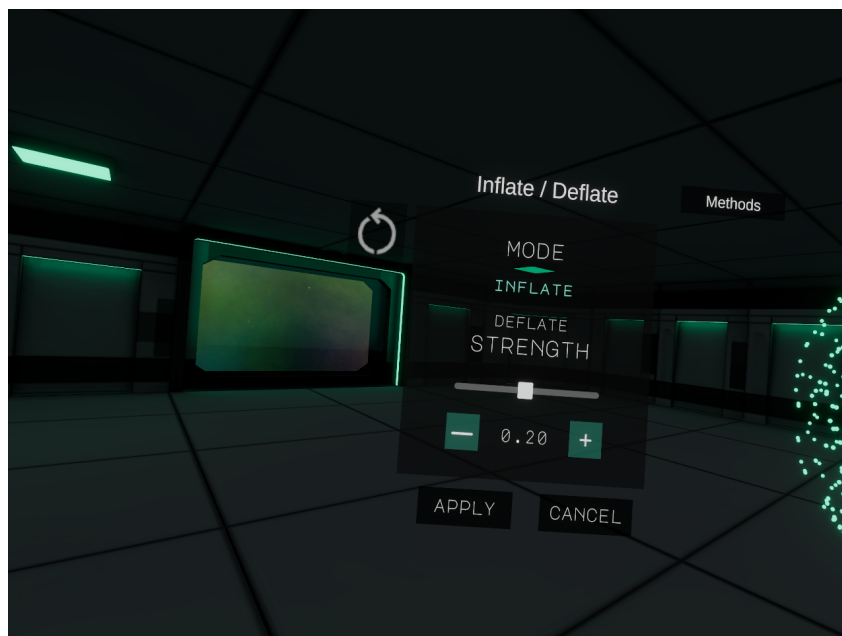
4.2.3 Ovládací panely a uživatelský workflow

Do klientské vrstvy byly doplněny panely pro volbu metody a ovládání metody `Inflate/Deflate`. Jejich účelem je převést akci uživatele ve VR na jednoznačný stav aplikace.

Základní práce s panely v ruce zůstává stejná jako v původní verzi aplikace. Nově bylo doplněno připnutí panelu do scény před uživatele, aby bylo možné měnit parametry bez držení panelu v ruce. Připnutí se aktivuje dvojklikem grab tlačítka na straně madla ovladače. Levý ovladač ovládá levý panel a pravý ovladač pravý panel. Přepnutí aktivní metody zároveň mění význam interakce s centry, ale samotné spuštění interakce zůstává pro obě metody shodné.

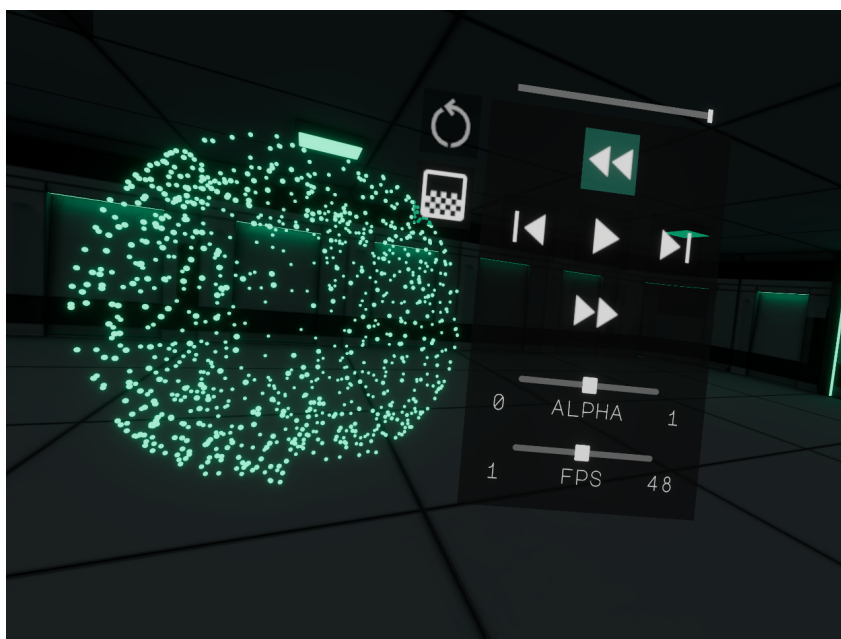
Workflow panelů ve VR je zachycen na obrázcích 4.2 až 4.11.

1. **připnutí panelu do scény.** Aktivuje se dvojklikem grab tlačítka na odpovídajícím ovladači. Levý ovladač připíná levý panel, pravý ovladač připíná pravý panel.



Obrázek 4.2 levý ovládací panel připnutý ve VR scéně.

Zdroj: vlastní zpracování.



Obrázek 4.3 pravý ovládací panel připnutý ve VR scéně.

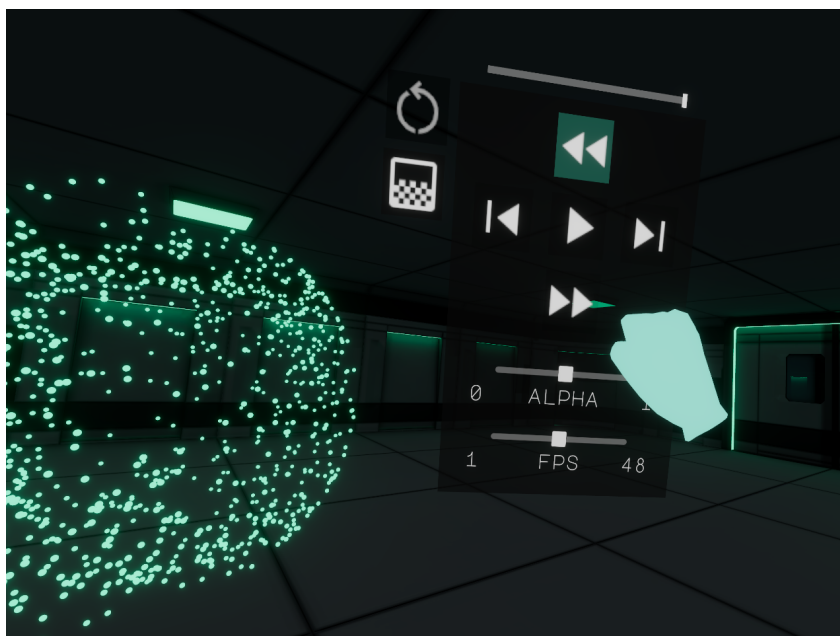
Zdroj: vlastní zpracování.

2. **přesun připnutého panelu.** Lze pomocí grabu posouvat nahoru, dolů, doprava nebo doleva kolem uživatele a jejich pozice se aktualizuje s pozicí kamery ve scéně. Panel si při pohybu zachovává fixní vzdálenost.



Obrázek 4.4 přesun připnutého panelu doleva pomocí grab tlačítka na controlleru.

Zdroj: vlastní zpracování.



Obrázek 4.5 přesun připnutého panelu doprava pomocí grab tlačítka na controlleru.

Zdroj: vlastní zpracování.

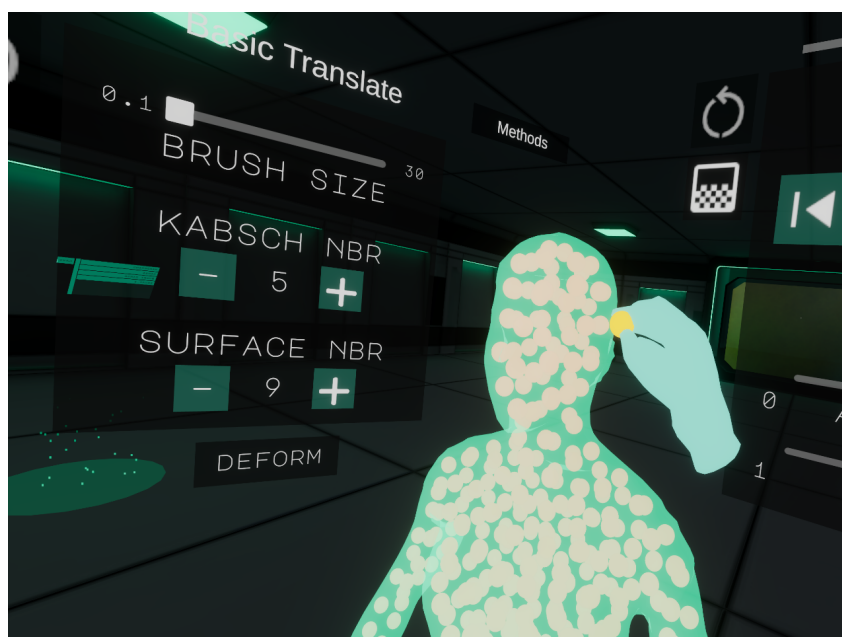
3. **volba aktivní metody.** Uživatel controllerem aktivuje tlačítko pro výběr metody a z rozbaleného panelu vybere Basic Translate nebo Inflate/Deflate.



Obrázek 4.6 rozbalený panel výběru aktivní metody.

Zdroj: vlastní zpracování.

4. **výběr centra.** U obou metod se centrum vybírá stejnou interakcí. U Basic Translate je povolen pohyb centra pro výpočet translace, u Inflate/Deflate se centrum pouze označí stisknutím spouště a použije se jako referenční bod deformace.



Obrázek 4.7 výběr a posun centra při metodě Basic Translate.

Zdroj: vlastní zpracování.



Obrázek 4.8 označení referenčního centra pro metodu Inflate/Deflate.

Zdroj: vlastní zpracování.

5. **nastavení Inflate/Deflate.** Uživatel zvolí režim Inflate nebo Deflate, nastaví sílu zásahu a sleduje náhled ovlivněné oblasti.



Obrázek 4.9 volba režimu Inflate nebo Deflate v panelu metody.

Zdroj: vlastní zpracování.

6. **potvrzení nebo zrušení operace.** Uživatel potvrzuje operaci a spustí deformaci, případně aktuální výběr zruší tlačítkem cancel nebo vrátí scénu do původního stavu tlačítkem reload vlevo vedle panelu.



Obrázek 4.10 potvrzení nastavení připravené operace.

Zdroj: vlastní zpracování.



Obrázek 4.11 spuštění výpočtu deformace.

Zdroj: vlastní zpracování.

Za přepínání aktivní metody odpovídá `MethodSelectionUI`. Stav panelu Inflate/-Deflate drží `InflateDeflateUI` a konkrétní hodnoty mění pomocné UI prvky.

4.3 Datové rozhraní mezi klientem a výpočtem

Mezi klientskou vrstvou a výpočetním jádrem je zavedeno explicitní datové rozhraní. Klient nepředává do výpočtu přímo objekty Unity scény, ale vytvoří request, tedy datový požadavek popisující zamýšlenou operaci. Výpočetní jádro po dokončení vrací `EditOperationResult`, který informuje o výsledku operace.

4.3.1 Request a výsledek operace

Request a výsledek operace tvoří datovou hranici mezi klientskou VR vrstvou a výpočetním jádrem. Klient zpracuje vstup uživatele, ale do výpočtu nepředává objekty Unity scény. Místo toho vytvoří datový požadavek s informacemi nutnými pro zvolenou metodu.

Zdrojový kód 4.1 základ editačního requestu

```
1 public abstract class EditOperationRequest
2 {
3     public MethodKind MethodKind { get; }
4     public string SequenceId { get; set; }
5     public int FrameIndex { get; set; }
6 }
```

Konkrétní metody používají vlastní potomky tohoto requestu. U Basic Translate obsahují vybrané centrum a cílovou pozici. U Inflate/Deflate se používá referenční centrum, režim operace a hodnota strength.

Zdrojový kód 4.2 výsledek editační operace

```
1 public sealed class EditOperationResult
2 {
3     public bool Succeeded { get; set; }
4     public MethodKind Method { get; set; }
5     public int FrameIndex { get; set; }
6     public string ErrorMessage { get; set; }
7 }
```

Tento výsledek neobsahuje upravená centra ani vrcholy meshe. Slouží pouze jako informace o úspěchu operace, použité metodě, framu a případné chybě. Upravená data zůstávají v runtime datech sekvence, ze kterých klient po dokončení operace aktualizuje scénu.

VR controller -> klientská interakce -> request -> core výpočet
-> EditOperationResult -> aktualizace scény z runtime dat

4.3.2 Runtime kontext sekvence

Vedle requestu potřebuje výpočet také aktuální stav sekvence. Request popisuje konkrétní akci uživatele, zatímco runtime kontext popisuje data, nad kterými se má tato akce provést. Díky tomu není nutné do requestu vkládat celý mesh, seznam center ani další data sekvence.

Zdrojový kód 4.3 zjednodušená struktura runtime kontextu

```

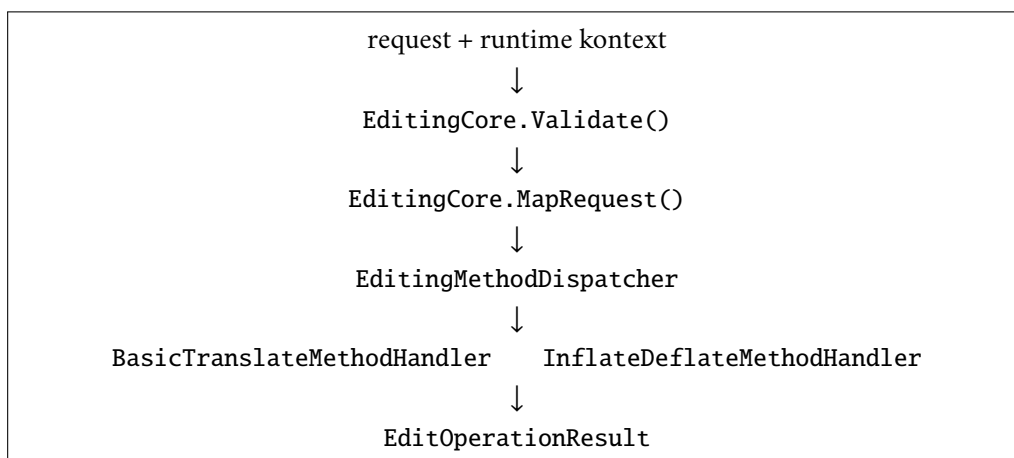
1 public sealed class EditingRuntimeContext
2 {
3     public int FrameIndex { get; init; }
4     public Vector3[] Centers { get; init; }
5     public Vector3[] Vertices { get; init; }
6     public int[] Triangles { get; init; }
7 }

```

Kontext vzniká z načtené sekvence a runtime stavu klienta, ale výpočetní vrstva z něj dostává pouze datovou reprezentaci. Nová editační metoda tak nemusí řešit načítání scény, práci s UI ani získávání hodnot z ovladačů.

4.4 Core vrstva a editační metody

Core vrstva provádí vlastní výpočet a nepracuje přímo s objekty Unity scény. Jejím vstupním bodem je `EditingCore`, který přijme request a runtime kontext, ověří vstupní data, převede request na interní vstup metody a předá jej odpovídajícímu handleru. Výsledek se poté vrací klientské vrstvě jako `EditOperationResult`. Výběr konkrétní metody neprovádí klientská třída `Sequence`. Tato odpovědnost je přesunuta do `EditingMethodDispatcher`, který podle hodnoty `MethodKind` vybere registrovaný handler. Díky tomu klient nemusí znát vnitřní složení jednotlivých algoritmů a nové metody lze připojovat přes stejný princip.



Obrázek 4.12 průchod requestu společným výpočetním jádrem.

Zdroj: vlastní zpracování.

Zdrojový kód 4.4 zjednodušený průchod requestu přes EditingCore

```

1 public EditOperationResult Execute(
2     EditOperationRequest request,
3     SequenceRuntimeContext context)
4 {
5     var validation = Validate(request);
6     if (!validation.IsValid)
7         return CreateFailedResult(request, validation);
8
9     var input = MapRequest(request, context);
10    var result = dispatcher.Dispatch(request.MethodKind,
11        input);
12
13    return CreateEditOperationResult(request, result);
14 }
```

4.4.1 Basic Translate

Basic Translate je základní metoda převzatá z původního VR editoru. V projektu „tvm-VR-2“ byla oddělena od scénové logiky a napojena přes společné výpočetní jádro.

Uživatel ve VR posune vybrané centrum na cílovou pozici. Klientský vstup se převádí na BasicTranslateRequest a v jádře aplikace dále na interní vstup metody. Samotný výpočet zůstává postavený na původním principu a skládá se ze tří kroků:

- GaussianCenterDeformer spočítá lokální deformaci center,
- KabschSequenceDeformer propaguje změnu přes sekvenci,
- NeighborhoodSurfaceDeformer obnovuje povrch podle upravených center.

Metoda zachovává původní chování, ale výpočet už není řízen přímo ze scénové třídy.

4.4.2 Inflate/Deflate

Inflate/Deflate je metoda napojená z projektu „tvm-editing-master“. Na rozdíl od Basic Translate nebyla pouze přesunuta z původní aplikace, ale musela být připojena přes samostatný adaptér.

Uživatel ve VR vybírá referenční centrum, režim deformace a sílu zásahu `strength`. Klientský vstup se převádí na `InflateDeflateRequest` a v jádře aplikace dále na interní vstup metody.

Výpočet je oproti Basic Translate napojen přes adaptér a probíhá v několika krocích:

- `TvmEditingMasterInflateDeflateAdapter` připraví data framu, centrum a efektory,
- `MeshEditor` zorganizuje volání převzatých výpočetních částí,
- `AffinityCenterDeformation` deformuje centra,
- `KabschTransformPropagation` propaguje transformace přes sekvenci,
- `CustomSurfaceDeformation` obnovuje povrch.

Díky tomuto napojení lze metodu z externího projektu obsloužit stejným tokem jako ostatní editační metody, bez přenášení jejich vnitřních detailů do VR klienta.

Architektura je zároveň připravena i na metodu `Looping`, která zatím není implementována, ale má v systému vyhrazené místo pro budoucí napojení stejným způsobem jako ostatní metody.

4.5 Sběr efektorů pro Inflate/Deflate

Efektory nejsou zadávány přímo, ale jsou sestaveny automaticky z afinit mezi centry. Afinita určuje, jak silně spolu dvě centra souvisí z hlediska tvaru a pohybu sekvence. Z cache se nejprve načte matice afinit. Pro zvolené referenční centrum se vezme odpovídající řádek matice a centra se seřadí podle hodnoty afinity. Centra s nejvyšší afinitou tvoří pohyblivé efektory. Část dalších center s nižší afinitou je použita jako stabilizační efektory s nulovou translací.

algoritmus sestavení efektorů

vstup referenční centrum c_r , matice afinit A , hodnota `strength`, režim metody
výstup seznam efektorů s odpovídajícími translacemi

- 1 načti řádek matice afinit $A[c_r]$,
- 2 seřaď ostatní centra podle afinity k c_r ,
- 3 vyber pohyblivé efektory C_m s vysokou afinitou,
- 4 vyber stabilizační efektory C_f s nižší afinitou,
- 5 spočítej afinitně vážený střed o pro oblast C_m ,
- 6 pro každé centrum $c_i \in C_m$ urči směr od středu oblasti o k danému centru,
- 7 urči váhu podle relativní hodnoty afinity,
- 8 vytvoř translaci podle hodnoty `strength` a režimu `Inflate/Deflate`,
- 9 pro centra C_f přidej nulovou translaci.

Obrázek 4.13 zjednodušený postup sběru efektorů implementovaný v `TvmEditingMasterInflateDeflateAdapter`.

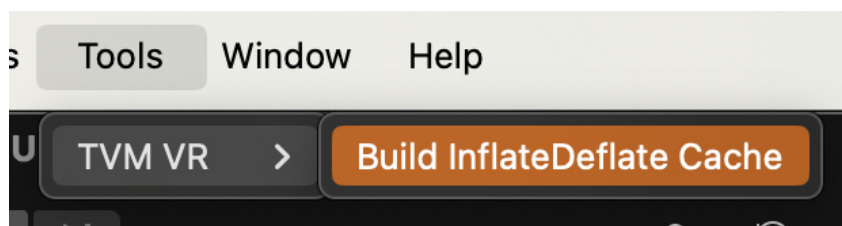
Zdroj: vlastní zpracování.

Výsledkem sběru je seznam indexů center a jim odpovídajících transformací. Slouží jako vstup pro metodu z projektu „tvm-editing-master“, která dopočítá deformaci ostatních center a následně deformaci povrchu.

4.6 Cache

Metoda Inflate/Deflate používá výpočetně náročná data, která závisí pouze na původně načtené sekvenci. Jedná se hlavně o afinity mezi centry a pomocná data pro deformaci povrchu a propagaci transformací. Proto byla doplněna cache, aby se tyto části nemusely počítat při každé editaci znovu.

Cache se sestavuje mimo běžný runtime aplikace pomocí nástroje v editoru Unity, jehož umístění ukazuje obrázek 4.14. Při aktuálním počtu sekvencí trvá její vytvoření přibližně hodinu a půl, protože se předpočítává pro všechny potřebné sekvence najednou. Výsledek se ukládá jako binární soubory do složky `Assets/StreamingAssets`, odkud jej při editaci načítá adaptér metody.



Obrázek 4.14 nástroj pro sestavení cache v editoru Unity.

Zdroj: vlastní zpracování.

Hlavními částmi cache jsou `manifest.bin` s informacemi pro ověření kompatibility, `affinity.bin` s maticí afinit, `structural.bin` se sousedními centry pro propagaci a framové soubory ve složkách `surface/` a `kabsch/`, které ukládají pomocné váhy pro povrchovou deformaci a Kabsch propagaci.

Postup:

1. offline nástroj předpočítá data pro zvolenou sekvenci,
2. výsledek uloží do `StreamingAssets`,
3. adaptér při editaci zkusí cache načíst,
4. pokud cache existuje, použije připravený kontext,

5. pokud cache chybí, data se dopočítají za běhu.

Cache nemění výsledek deformace. Pouze nahrazuje opakovaný výpočet mezivýsledků jejich načtením z připravených souborů. Klientské VR workflow tím zůstává beze změny.

Výsledky a omezení

5

5.1 Výsledky implementace

Výsledkem práce je funkční projekt „tvm-VR-2“, který zachovává klientské VR prostředí původní aplikace, ale současně zavádí čitelnější oddělení odpovědností mezi klientskou vrstvou, datovou hranicí a výpočtním jádrem. V rámci implementace se podařilo převést původní základní metodu do nového jádra, připojit metodu Inflate/Deflate z projektu „tvm-editing-master“, doplnit potřebný request model a navrhnout klientské workflow pro její použití přímo ve VR prostředí.

Praktickým přínosem je zejména to, že aplikace již není vázána na jeden úzký tok editace, ale umožňuje obsloužit více metod přes stejný architektonický princip. Současně se podařilo zavést i podpůrné techniky, například cache execution contextu, které zlepšují použitelnost integrované metody při opakovaném volání.

5.2 Ověření chování aplikace a uživatelského workflow

Při ověřování bylo důležité potvrdit správné načítání sekvencí, průchod uživatelského vstupu do requestu a použitelnost obou hlavních metod ve VR workflow. Základní metoda zůstala po refaktoringu zachována, zatímco metoda Inflate/Deflate byla začleněna do nové architektury i do uživatelského rozhraní.

Cílem ověření bylo zkontrolovat, zda je možné sekvenci načíst, vybrat centrum, pracovat s ovládacími panely a spustit dostupné editační metody bez zásahu do interní logiky aplikace.

Součástí ověření bylo také neformální vyzkoušení aplikace přímo ve VR prostředí na neznalém uživateli. Testy byly provedeny na dvou uživateli a oba potvrdili, že si během krátké doby osvojili ovládání aplikace a že uživatelské rozhraní je v určitých mezích intuitivní a snadno použitelné. Při tomto ověření se jako důležité ukázalo připnutí panelů do prostoru před uživatelem, ovládání panelů pomocí kontrolerů a odlišení významu interakce s centrem pro jednotlivé metody. Ověření

potvrdilo použitelnost nového workflow, i když pro přesnější hodnocení ergonomie by bylo vhodné provést testování s větším počtem uživatelů.

5.3 Omezení aktuálního řešení

Vedle přínosů je nutné uvést i omezení. Projekt vznikl jako praktický prototyp pro prezentaci výsledků výzkumu a neřeší všechny možné směry dalšího vývoje. Část XR infrastruktury byla během migrace udržena pomocí přechodových kompatibilitních kroků, zejména v oblasti vstupních akcí a některých scénových vazeb. Podobně ani výkonnost výpočetně náročnější metody nelze považovat za definitivně vyřešenou; cache sice pomáhá, ale nepředstavuje plnou náhradu za hlubší optimalizaci celé externí pipeline.

Dalším omezením je rozsah implementovaných uživatelských scénářů. Aplikace je navržena pro prezentaci a experimentální editaci vybraných metod, nikoli jako plně obecný editor všech možných deformací sekvencí polygonálních sítí. Některé infrastrukturní a uživatelské části proto zůstávají otevřené pro další dopracování.

5.4 Použití AI při práci

Při vývoji a při přípravě dokumentace byly využity nástroje založené na umělé inteligenci jako podpůrný prostředek pro analýzu zdrojových kódů, návrh struktury textu a formulaci technických poznámek. Samotný návrh architektury, implementační rozhodnutí, ověřování chování aplikace i konečná podoba textu však zůstaly pod přímou kontrolou autora.

Cílem práce bylo navrhnout a implementovat novou VR aplikaci pro prezentaci a praktické ovládání vybraných metod editace sekvencí polygonálních sítí. Tento cíl byl splněn vytvořením projektu „tvm-VR-2“, který navazuje na původní VR editor, ale zavádí přehlednější oddělení uživatelské interakce, datového rozhraní a výpočetního jádra.

Hlavním přínosem práce je sjednocený způsob připojení editačních metod. Původní metoda Basic Translate byla převedena do nové struktury a metoda Inflate/Deflate z projektu „tvm-editing-master“ byla napojena přes samostatný adaptér. VR klient tak může obě metody používat přes stejný princip requestu, runtime kontextu a výsledné odpovědi, aniž by závisel na jejich vnitřní implementaci.

Součástí výsledku je také nové VR workflow pro výběr metody, práci s centry, ovládání Inflate/Deflate a manipulaci s panely v prostoru. Pro výpočetně náročnější části byla doplněna cache, která nemění výsledek deformace, ale zrychluje získání potřebných mezivýsledků. Aktuální řešení zůstává prototypem a ponechává prostor pro další optimalizaci, sjednocení některých částí VR ovládání a širší uživatelské testování. Přesto práce ukazuje, že editaci nekonzistentních sekvencí polygonálních sítí lze uživateli zpřístupnit přímo ve virtuální realitě nad architekturou, která je srozumitelná a dále rozšiřitelná.

Seznam použité literatury

1. KÁČERKOVÁ, Zuzana. *Systém pro editaci sekvencí trojúhelníkových sítí s časově proměnlivou konektivitou* [online]. 2023. [cit. 2026-05-22]. Dostupné z: <https://dspace.zcu.cz/items/bd4f62e3-6fe3-4670-ad4d-12e370831822>.
2. HACHA, Filip. *TVM Editing* [online]. 2024. [cit. 2026-05-22]. Dostupné z: <https://gitlab.com/hachaf/tvm-editing>.